AN EXPLAINER, IN 15 SLIDES

How to price a derivative and measure its risk — by rolling dice.

Monte Carlo simulation, from the model to the risk number, on a 16 MiB computer with no operating system.

Every figure here was computed on the machine, and checked against an exact formula.

THE QUESTION

Two things a trading desk needs to know.

Price: what is this option worth today?

Risk: if the market moves against me, how much could I lose — and how likely is that?

An *option* here is the right to buy a stock at a fixed price (the strike) on a future date. It pays off only if the stock ends up above the strike.

STEP 1 — THE MODEL

A stock price is a random walk with drift.

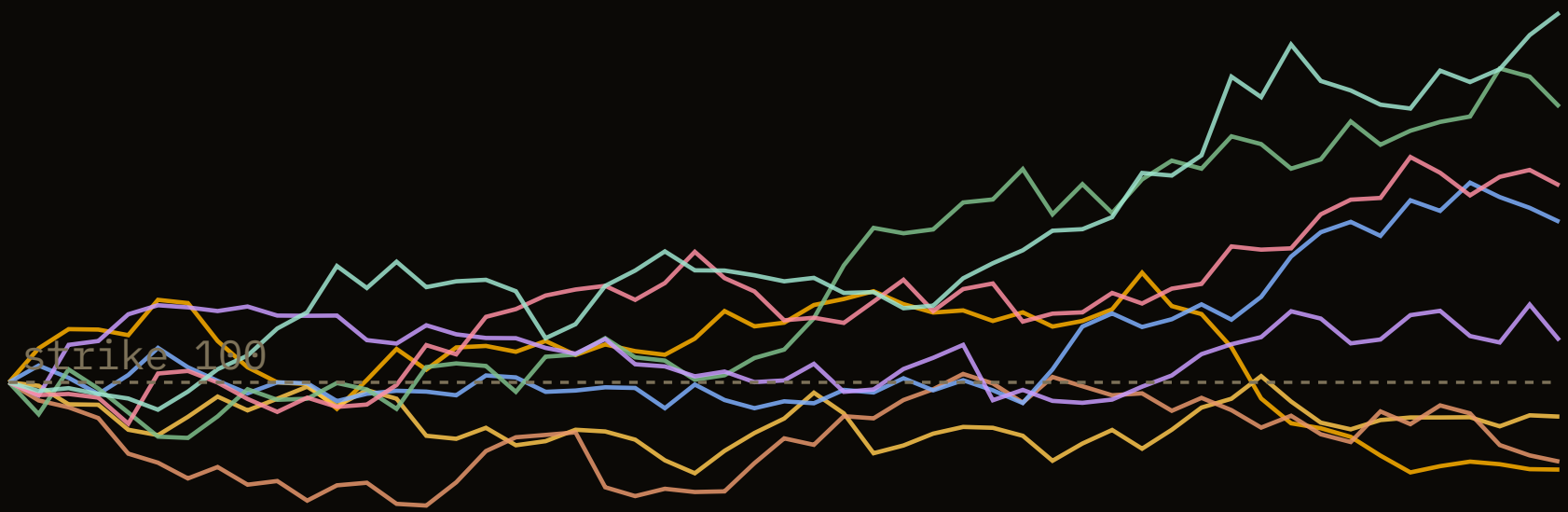
The standard model is Geometric Brownian Motion. Over a small step of time, the price is nudged by a fixed drift and a random shock:

$$S(t+dt) = S(t) \cdot \exp\left((r - \tfrac{1}{2}\sigma^2)\cdot dt + \sigma\cdot\sqrt{dt} \cdot Z \right)$$

r is the interest rate, σ the volatility, and Z a random draw from a bell curve. Chain these steps together and you get one possible future for the price.

STEP 1 — THE MODEL

Eight futures, drawn by the machine.



Each line is one random future. We roll millions of them.

STEP 2 — THE PRICE

Average the payoff, and discount it.

For each simulated future, the option pays max(final price − strike, 0). Average that across every path, and bring it back to today's money:

```
price ≈ e^(−rT) · average of max(S_T − K, 0)
```

That is the whole method. The hard part is knowing when to trust the average.

STEP 2 — THE PRICE

More rolls, tighter answer.

1e3 samples → 8e6

Error shrinks like 1/√N: 100× the samples, 10× the accuracy.

STEP 3 — THE CHECK

For this option, the exact answer is known.

A simple European call has a closed form — the 1973 Black–Scholes formula. It is the ruler we measure the simulation against.

METHOD	PRICE
Black-Scholes (exact)	10.4506
Monte Carlo, 8M paths	10.4498 ± 0.0026

Within one standard error. The dice found the formula.

A FREE IMPROVEMENT

Antithetic variates: cancel the noise.

For every random shock Z you draw, also use its mirror −Z. When one path runs lucky, its twin runs unlucky, and the two errors partly cancel.

```
plain       10.4486 +- 0.0052
antithetic  10.4498 +- 0.0026    (4x less variance)
```

Same number of draws, half the error bar. Free accuracy from a symmetry.

STEP 4 — THE SENSITIVITIES

The Greeks: how the price reacts.

GREEK	VALUE	WHAT IT MEASURES
delta	0.637	move per $1 of stock
gamma	0.019	how delta itself moves
vega	0.375	per 1% of volatility
theta	−0.018	lost per day of time
rho	0.532	per 1% of interest rate

The simulation recovers delta (0.637) and vega (0.375) too, matching the formula.

STEP 5 — THE RISK

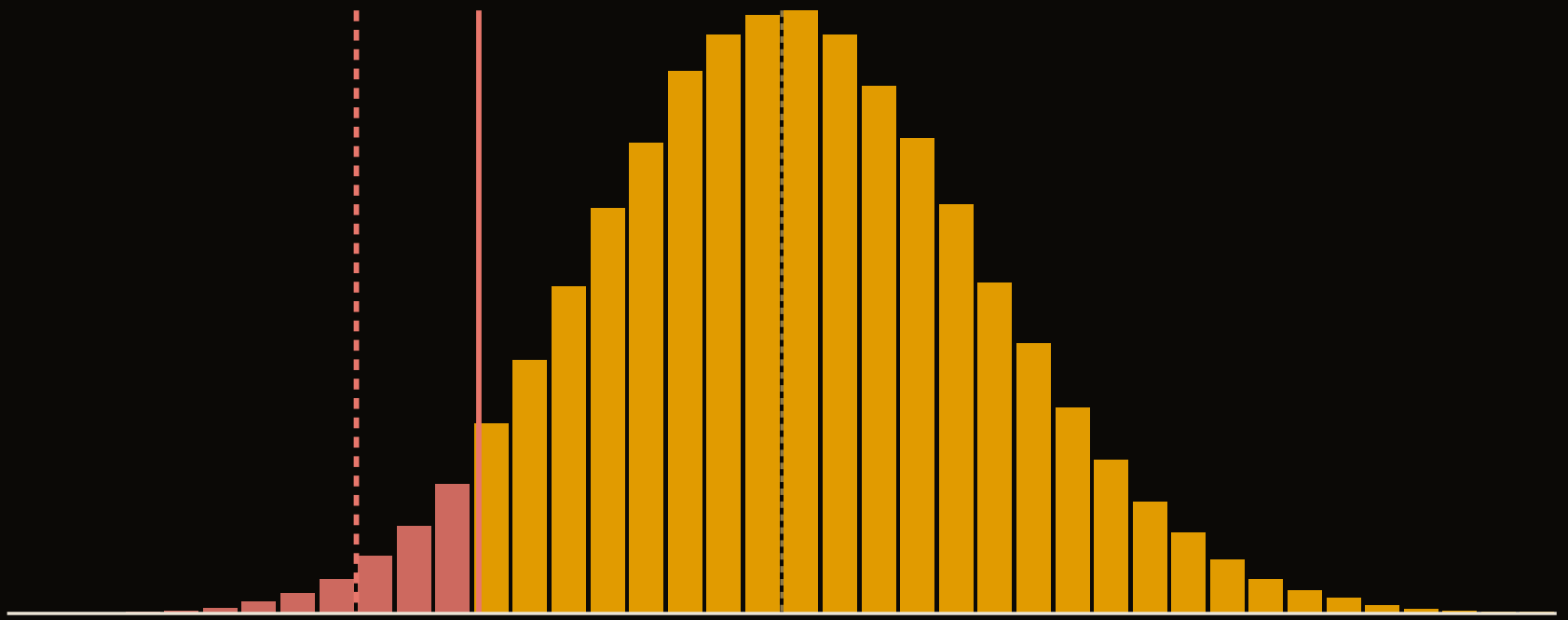
Risk is not one number. It is a distribution.

To measure risk, simulate the near future of a position — here, $1M of the stock over 10 trading days — and look at the whole spread of profits and losses.

Most outcomes cluster near zero. The question is how bad the bad tail gets.

STEP 5 — THE RISK

Value-at-Risk reads off the tail.



VaR95 = $62k means: on the worst 1 day in 20, you lose at least this.

STEP 5 — THE RISK

Expected Shortfall: how bad, when it is bad.

Value-at-Risk is a threshold. It says nothing about how far past it you can go. Expected Shortfall answers that — the *average* loss on the worst days.

VaR 95%	$62,327
VaR 99%	$87,507
Expected Shortfall 99%	$99,624

When the worst 1% happens, the average loss is worse than the VaR line.

THE WHOLE MACHINE

One unikernel, every service load-bearing.

Telegram	the instrument, in plain English
Firecrawl	live market data over HTTPS
Gemini	parse it into parameters
RDRAND	the random shocks (hardware entropy)
compute	simulate, price, measure
Redis	an immutable audit record
PNG + Telegram	the chart and the answer

WHY A UNIKERNEL

Determinism is the product.

A regulated risk number must be reproducible and auditable — every run logged, every run repeatable. That is not free: this platform silently computed wrong 1 in 4,000 times until we root-caused and fixed it.

And the randomness is real: every shock comes from the CPU's hardware entropy, proven against ent, FIPS 140-2 and dieharder. A unikernel has no /dev/random; this one makes its own.

OPEN SOURCE

The engine, the proofs, the walkthrough.

> github.com/tabibazar/unikernel-c

Monte Carlo pricing with variance reduction and Greeks, Value-at-Risk from the simulated distribution, and the hardware RNG it all runs on — a few hundred lines of C, no operating system, no math library.

Built on Return Infinity's BareMetal. Prices by Black-Scholes; the dice are the machine's own.